Киберфизическая система: Распределённый монитор банковских транзакций

Juan Pablo Sierra Useche (473836/4150C)
Задание — Киберфизические системы
Университет ИТМО · April 2026

Примечание: Документ написан на основе рабочих заметок о проектировании и реализации системы. Текст структурирован и адаптирован с помощью языковой модели, затем вручную добавлен в шаблон на `typst`.

Содержание

1. План . . . 3
1.1. Назначение системы . . . 3
1.2. Архитектура системы . . . 3
1.3. Описание уровней системы . . . 4
1.3.1. Уровень 1 — Физический . . . 4
1.3.2. Уровень 2 — Сбор и генерация данных . . . 4
1.3.3. Уровень 3 — Сеть и дейтаграммы . . . 4
1.3.4. Уровень 4 — Обработка . . . 5
1.3.5. Уровень 5 — Аналитический центр . . . 6
1.4. Алгоритмы вычислений . . . 6
1.4.1. Оценка аномалий (скользящее окно) . . . 6
1.4.2. Конвейер проверки целостности дейтаграмм . . . 6
1.5. Мониторинг . . . 7
1.6. Область применения . . . 7
2. Реализация . . . 8
2.1. Структура проекта . . . 8
2.2. Изменение формата дейтаграммы . . . 8
2.3. Отправитель . . . 8
2.3.1. Генератор транзакций . . . 9
2.3.2. Режим физического вмешательства . . . 9
2.4. Приёмник . . . 9
2.5. Дашборд . . . 10
2.6. Развёртывание . . . 10
2.7. Что ещё не сделано . . . 11

1. План

1.1. Назначение системы

Система воспроизводит минимальную банковскую инфраструктуру: два граничных вычислительных узла отправляют синтетические транзакции по зашифрованному каналу на центральный сервер, который проверяет, хранит и помечает аномалии.

Такая схема встречается в реальных финансовых IoT-решениях: ограниченные по ресурсам устройства генерируют чувствительные данные, данные должны оставаться конфиденциальными при передаче, а облачный центр выявляет аномалии. Реализация охватывает все пять уровней КФС.

1.2. Архитектура системы

Система построена по пяти уровням референсной модели КФС. Два узла на **Raspberry Pi Zero 2W** — физическая граница; дроплет **DigitalOcean** — аналитический центр. Вся связь между узлами и сервером идёт через **UDP-дейтаграммы**, каждая с криптографически защищённой полезной нагрузкой.

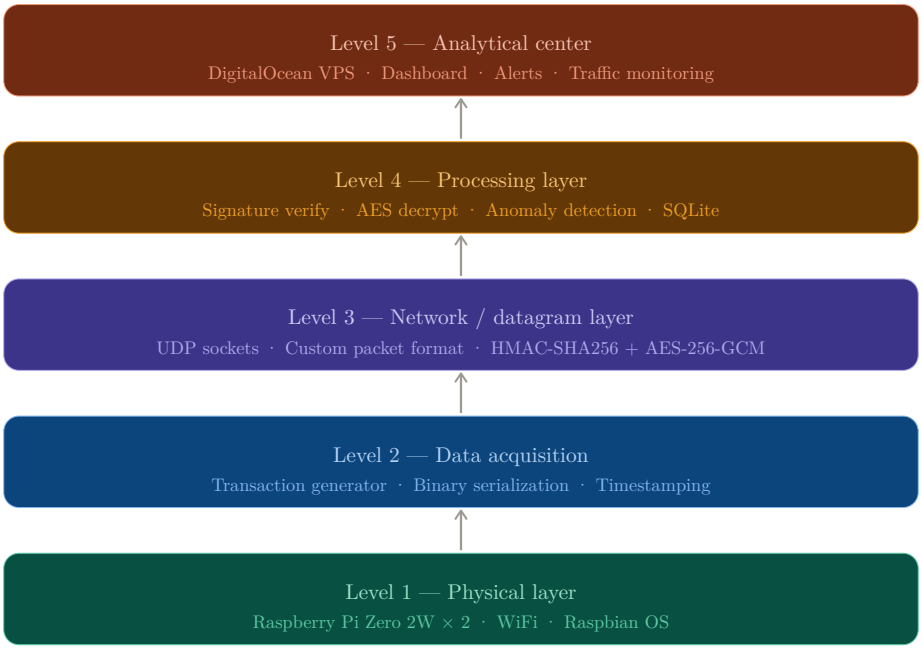

Рис. 1. Уровни КФС

1.3. Описание уровней системы

1.3.1. Уровень 1 — Физический

Физический уровень — два одноплатных компьютера **Raspberry Pi Zero 2W**, каждый с безголовой установкой **Alpine Linux**, чтобы не тратить лишние ресурсы. Связь с сетью — по Wi-Fi: либо через кампусную сеть, либо через мобильную точку доступа как запасной вариант.

Это самый ограниченный конец стека: слабый процессор, мало оперативной памяти, нет дисплея, всё управление — только по SSH. В данной симуляции они генерируют события вместо того, чтобы считывать реальные, и передают их наверх.

1.3.2. Уровень 2 — Сбор и генерация данных

На каждом узле работает скомпилированный **бинарник на Rust**, который генерирует синтетические банковские транзакции. Транзакция — это запись со следующими полями:

Поле	Описание
`node_id`	Идентификатор Raspberry Pi, с которого отправлена транзакция
`account_id`	Синтетический идентификатор банковского счёта
`amount`	Сумма транзакции в условных единицах
`tx_type`	Тип: пополнение, снятие, перевод
`timestamp`	Unix-время в микросекундах (`u64`)

Записи сериализуются через общий крейт Rust, который используется на обоих узлах и на сервере — единый бинарный формат по всей системе.

1.3.3. Уровень 3 — Сеть и дейтаграммы

Транзакции передаются через **UDP-сокеты** в дейтаграммах с фиксированной структурой:

```
[node_id:2B][seq:4B][timestamp:8B][nonce:12B][ciphertext:12B][hmac_tag:32B]
```

Обоснование полей:

- **`node_id (2B)`** — Двух байт хватает для адресации до `65 536` узлов; для двух Raspberry Pi в этом проекте — с большим запасом.
- **`seq (4B)`** — Монотонно возрастающий счётчик для каждого узла. 2^{32} значений перекроют любой реалистичный срок работы демо и позволяют серверу обнаружить атаку повторного воспроизведения.
- **`timestamp (8B)`** — Unix-время в формате `u64`, ровно 8 байт. Используется для измерения задержки и упорядочивания дейтаграмм, пришедших не по порядку.
- **`nonce (12B)`** — Стандартный размер nonce для GCM. AES-256-GCM строит ключевой поток из 96-битного nonce и 32-битного счётчика блока; поле в 12 байт соответствует этому требованию напрямую.

- **`ciphertext (12B)`** — Зашифрованная структура `Transaction`. AES-GCM — потоковый режим шифрования, поэтому длина шифртекста совпадает с длиной открытого текста. Структура ниже кодируется в `9B` данных плюс `3B` выравнивания — итого `12B`:

```rust
use std::mem::size_of;

#[repr(C)]
struct AccountId(u32);
#[repr(C)]
struct Amount(f32);
#[repr(u8)]
enum TransactionType {
    Deposit,
    Withdrawal,
    Transfer,
}
#[repr(C)]
struct Transaction {
    account_id: AccountId,
    amount: Amount,
    tx_type: TransactionType,
}
fn main() {
    println!("AccountId: {}", size_of::<AccountId>());              // 4B
    println!("Amount: {}", size_of::<Amount>());                    // 4B
    println!("TransactionType: {}", size_of::<TransactionType>());  // 1B
    println!("Transaction: {}", size_of::<Transaction>());          // 12B
}
```

- **`hmac_tag (32B)`** — Дайджест HMAC-SHA256, вычисленный по всем предшествующим полям дейтаграммы (кроме самого тега). Обеспечивает проверку целостности незашифрованных полей заголовка, которые AEAD не покрывает.

Схема шифрования: полезная нагрузка `Transaction` шифруется с помощью **AES-256-GCM** — схема AEAD, которая одновременно обеспечивает конфиденциальность и аутентичность шифртекста. Внешний **HMAC-SHA256** покрывает всю обёртку дейтаграммы, включая открытые метаданные (`node_id`, `seq`, `timestamp`): любое вмешательство в незашифрованный заголовок также будет обнаружено на сервере.

1.3.4. Уровень 4 — Обработка

Серверный конвейер обработки работает в следующем порядке:

1. **Проверка целостности:** верификация тега HMAC-SHA256; любая дейтаграмма, не прошедшая проверку, отбрасывается до дальнейшей обработки.
2. **Расшифровка:** AES-256-GCM расшифровывает шифртекст и проверяет тег GCM.
3. **Десериализация:** восстановление структуры `Transaction` через общий serde-крейт.
4. **Хранение:** проверенная транзакция сохраняется в базу данных.
5. **Обнаружение аномалий:** транзакция оценивается по скользящему окну из последних **50 записей** для каждого узла. Применяются две эвристики:

- **Проверка скорости:** флаг выставляется, если число транзакций в коротком временном окне превышает заданный порог (обнаружение всплеска).
- **Проверка суммы:** флаг выставляется, если сумма транзакции отклоняется от среднего по окну больше чем на $\pm 2\sigma$ (обнаружение выброса).

Помеченные транзакции сохраняются с маркером аномалии, а не отбрасываются — так сохраняется полный журнал.

1.3.5. Уровень 5 — Аналитический центр

Аналитический центр — дроплет **DigitalOcean**, на котором размещены база данных и лёгкий веб-дашборд. Вся инфраструктура описана в **Terraform**:

- Правила файрвола для входящего UDP (приёмник дейтаграмм) и HTTPS (дашборд);
- Инъекция SSH-ключа для безопасного доступа к управлению;
- Provisioning дроплета с воспроизводимой конфигурацией.

Дашборд показывает живую ленту входящих транзакций: последние записи, помеченные аномалии и статистику по узлам, с обновлением каждые несколько секунд.

1.4. Алгоритмы вычислений

1.4.1. Оценка аномалий (скользящее окно)

Пусть $W = \{a_1, a_2, ..., a_N\}$ — суммы последних $N = 50$ транзакций для данного узла. Определим:

$$\mu_W = \frac{1}{N} \sum_{\{i=1\}}^{\{N\}} a_i, \quad (1)$$

$$\sigma_W = \sqrt{\frac{1}{N-1} \sum_{\{i=1\}}^{\{N\}} (a_i - \mu_W)^2} (2)(3)$$

Входящая транзакция с суммой a помечается как аномальная, если:

$$|a - \mu_W| > 3 \cdot \sigma_W$$

Порог k настраивается. Аномалии скорости обнаруживаются подсчётом транзакций, временные метки которых попадают в скользящее 60-секундное окно, с последующим сравнением с настраиваемым лимитом всплеска.

1.4.2. Конвейер проверки целостности дейтаграмм

```
Transaction → [AES-256-GCM encrypt] → ciphertext → [HMAC-SHA256 sign] → datagram
```

[1]Источник формулы

[2]Источник формулы

[3]Используется исправленное стандартное отклонение, поскольку оно корректирует смещение при оценке по выборке и даёт более точный результат для небольшого окна.

На приёмнике конвейер работает строго в обратном порядке: сначала верификация HMAC, затем расшифровка GCM. Сбой на любом этапе — дейтаграмма молча отбрасывается и попадает в лог.

1.5. Мониторинг

Дашборд показывает следующие метрики в реальном времени:

Метрика	**Описание**
Лента транзакций	Хронологический список последних N проверенных транзакций
Лента аномалий	Транзакции, помеченные одной из эвристик, с указанием причины
Сигнал узла	Время последней активности каждого узла; статус «устаревший» при > 30 с
Пропускная способность	Транзакций в секунду, вычисляется за 10-секундное окно
Доля отброшенных	Отношение отброшенных дейтаграмм (неверный HMAC/GCM) к общему числу

Все метрики обновляются при каждом цикле обновления дашборда (примерно каждые 3 секунды). Серверные логи хранят полную историю событий для последующего анализа.

1.6. Область применения

Система проверяет только криптографическую корректность транзакции. Проверки баланса и доступности средств вне области применения.

2. Реализация

В этом разделе описывается система в том виде, в каком она реально построена, включая решения, которые отличаются от плана.

2.1. Структура проекта

Rust-воркспейс разделён на три бинарника и одну общую библиотеку:

Крейт / файл	Роль
`disgrams`	Общая библиотека: формат пакетов, криптография, типы транзакций
`sender`	Работает на каждом Raspberry Pi; генерирует и отправляет дейтаграммы
`receiver`	Работает на дроплете DigitalOcean; проверяет, расшифровывает, хранит
`cps-bank-dashboard.py`	HTTP-сервер на Python; читает SQLite-базу и отдаёт живой вид

2.2. Изменение формата дейтаграммы

В плане внешний HMAC-SHA256 защищал незашифрованные поля заголовка. При реализации от этого отказались в пользу более простого подхода: байты заголовка передаются как **Additional Authenticated Data (AAD)** прямо в AES-256-GCM. Тег аутентификации GCM покрывает и шифртекст, и AAD, поэтому любое изменение открытого заголовка делает расшифровку невозможной — без отдельного прохода HMAC.

Итоговая структура:

```
[node_id:2B][seq:4B][timestamp:8B][nonce:12B][ciphertext:9B][gcm_tag:16B]
```

Общий размер пакета — 51 байт. Внешнего HMAC из плана нет; 16-байтный тег GCM берёт на себя обе роли. Структура `Transaction` сериализуется вручную (поля в big-endian, без C-выравнивания), поэтому укладывается в 9 байт, а не в 12, как предполагалось при планировании.

2.3. Отправитель

Бинарник отправителя читает конфигурацию из переменных окружения: `CIPHER_KEY` (32-байтный ключ в hex), `NODE_ID`, `TARGET_HOST` и `TARGET_PORT`. Отсутствие `CIPHER_KEY` — фатальная ошибка при запуске; всё остальное имеет безопасные значения по умолчанию.

Счётчик последовательности хранится в текстовом файле (`.sender_sequence_counter_{node_id}`) с правами только для владельца (`0600`). Загружается при старте и записывается после каждой успешной отправки — счётчик переживает перезапуски без риска атаки повторного воспроизведения.

2.3.1. Генератор транзакций

Суммы и интервалы отправки формируются с помощью засеянного линейного конгруэнтного генератора (LCG). Свой LCG вместо зависимости нужен для кросс-компиляции под `armv6` (цель — Raspberry Pi Zero 2W) без крейтов, которые могут не поддерживать эту платформу.

Нормальные транзакции генерируются каждые 3,5–7 секунд с суммами от 10 до 500 условных единиц. Каждая восьмая транзакция — подозрительный выброс с суммой от 5 001 до 25 000 единиц, чего достаточно, чтобы сработал алерт по сумме на дашборде.

Аномалии скорости инжектируются паттерном всплеска: после каждых 42 обычных транзакций отправитель выдаёт 10 пакетов с интервалом 20–80 мс, затем делает паузу на 65 секунд и возвращается к нормальному темпу. Это создаёт повторяемый всплеск скорости, который мониторинговый уровень может наблюдать.

2.3.2. Режим физического вмешательства

Отправитель проверяет наличие файла, путь к которому задаётся переменной окружения `TAMPER_FILE`. Когда этот файл существует, каждый исходящий пакет заменяется фиксированной подозрительной транзакцией (`account_id=999 999`, `amount=25 000`, `tx_type=Transfer`). Файл можно будет впоследствии привязать к GPIO-пину, подключённому к кнопке или геркону, — это симулирует событие физического вмешательства на нижнем уровне. Пока что срабатывание делается программно; реальная разводка GPIO — часть второго задания.

2.4. Приёмник

Приёмник — асинхронный бинарник на Tokio. При запуске он подключается к SQLite-базе (через `sqlx`) и генерирует случайный 32-байтный AES-ключ для каждого настроенного `node_id`, сохраняя его через `INSERT OR IGNORE`, чтобы ключи не менялись при перезапусках. Ключ каждого узла выводится в stdout — оператор копирует его в переменные окружения соответствующего отправителя.

Для каждой входящей UDP-дейтаграммы приёмник:

1. Читает первые два байта, чтобы определить узел-отправитель.
2. Получает ключ этого узла из базы данных.
3. Вызывает `decrypt_packet`, который проверяет тег GCM (покрывающий заголовок и шифртекст) и расшифровывает полезную нагрузку.
4. В случае успеха вставляет транзакцию в `received_transactions`. В случае ошибки записывает её в лог и отбрасывает пакет.

Таблица `received_transactions` имеет ограничение `UNIQUE (node_id, seq)`, поэтому повторно воспроизведённая дейтаграмма молча отклоняется на уровне вставки.

2.5. Дашборд

Дашборд — один Python-скрипт, отдающий минималистичную HTML-страницу по HTTP. При каждом запросе он читает из SQLite-базы напрямую и обновляется каждые 5 секунд. При каждом рендере проверяются два условия для алертов:

- **Большая сумма:** любая транзакция в последних 25 записях с `amount ≥ 5 000`.
- **Неактивный узел:** любой узел, последняя транзакция которого старше 60 секунд.

Оценка аномалий на основе скользящего окна σ из плана к дашборду пока не подключена. Функция `get_last_fifty_trans` в кодовой базе приёмника есть, математика правильная, но результат пока не передаётся на дашборд и не сохраняется как маркер аномалии.

2.6. Развёртывание

Три systemd-сервиса управляют запущенными процессами: `cps-bank-receiver`, `cps-bank-dashboard` и `cps-bank-sender`. Каждый работает под отдельным пользователем `cpsbank` с ограниченными привилегиями (`NoNewPrivileges`, `ProtectSystem`, `ProtectHome`, `PrivateTmp`). Приёмник и дашборд используют одну базу данных по пути `/var/lib/cps-bank/receiver.db`.

Облачная инфраструктура описана в Terraform в директории `infra/terraform/`. Один `terraform apply` поднимает весь стек:

1. **Дроплет:** инстанс Ubuntu 24.04 `s-1vcpu-1gb` в регионе `nyc1`, инициализируемый через шаблон `cloud-init`, который устанавливает бинарники и регистрирует systemd-сервисы.
2. **SSH-ключ:** публичный ключ оператора загружается в DigitalOcean и инжектируется в дроплет при создании — пароль не устанавливается никогда.
3. **Файрвол:** создаются три правила для входящего трафика:
   - SSH (TCP 22) — только с адресов из переменной `admin_cidrs`.
   - Приёмник (UDP 9876) — только с адресов из `pi_cidrs`, то есть IP-диапазонов Raspberry Pi-узлов. Дейтаграмма, пришедшая с адреса вне этого списка, отбрасывается файрволом до того, как доберётся до процесса.
   - Дашборд (TCP 8080) — только с адресов из `admin_cidrs`.

   Весь исходящий трафик не ограничен.
4. **Вывод:** после `apply` Terraform печатает публичный IP дроплета, который затем копируется в переменную `TARGET_HOST` отправителя на каждом Raspberry Pi.

Список разрешённых IP существует потому, что адреса Raspberry Pi фиксированы и известны заранее. Блокировать неизвестные источники на уровне файрвола до того, как пакет достигнет процесса, — бесплатное усиление безопасности без каких-либо изменений в приложении.

2.7. Что ещё не сделано

GPIO-разводка не готова. Физические кнопки и светодиоды из плана — 2 кнопки, 4 светодиода, резисторы, провода — всё ещё не подключено. Механизм `TAMPER_FILE` пока выполняет роль программной заглушки; реальная разводка переносится на второе задание.

Сетевое взаимодействие в кампусной сети ИТМО оказалось сложнее, чем ожидалось. Проблемы с сертификатами и синхронизацией NTP на Raspberry Pi Zero 2W потребовали ручного вмешательства, поэтому сквозное тестирование между физическими узлами и дроплетом DigitalOcean ещё не завершено.